

CyberX Forensics Platform

Complete Installation & Setup Guide

Windows | Linux | macOS

1. Prerequisites

1.1 Python 3.10+

Python is required to run the FastAPI backend.

```
# Ubuntu / Debian
sudo apt install python3 python3-pip python3-venv

# macOS (Homebrew)
brew install python

# Windows
# Download from: https://www.python.org/downloads/
# Tick "Add Python to PATH" during install
python --version # verify
```

1.2 Node.js 18+

Node.js runs the React/Vite frontend.

```
# Ubuntu / Debian
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# macOS
brew install node

# Windows: https://nodejs.org/
node --version # verify
```

1.3 Git

```
sudo apt install git # Linux
# Windows: https://git-scm.com/download/win
```

1.4 Java 17+ (for Ghidra binary analysis)

```
sudo apt install default-jdk # Linux
# Windows: https://adoptium.net/
java -version # verify
```

2. Project Setup

2.1 Clone the Repository

```
git clone https://github.com/your-username/cyberx-forensics.git
cd cyberx-forensics
```

2.2 Backend: Python Virtual Environment

```
# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt

# Windows
python -m venv .venv
.venv\Scripts\activate
pip install -r backend\requirements.txt
```

2.3 Frontend: Node Packages

```
cd frontend
npm install
cd ..
```

2.4 Environment Variables (.env)

Create backend/.env and add your API keys:

```
MALWAREBAZAAR_API_KEY=your_key
VIRUSTOTAL_API_KEY=your_key
ABUSEIPDB_API_KEY=your_key
SHODAN_API_KEY=your_key
ALIENVAULT_API_KEY=your_key
```

NOTE: Free keys: virustotal.com | abuseipdb.com | shodan.io | otx.alienvault.com

3. System Tool Installation - Linux

3.1 One-Command Install

```
sudo apt update && sudo apt install -y \  
exiftool tshark steghide john sleuthkit python3-dev libssl-dev
```

3.2 Volatility 3 (Memory Forensics)

```
pip install volatility3  
vol --help # verify
```

3.3 Ghidra (Binary Reverse Engineering)

Ghidra is bundled in plugins/ghidra_12.1.2_PUBLIC/

```
# Make executable  
chmod +x plugins/ghidra_12.1.2_PUBLIC/support/analyzeHeadless  
  
# Verify  
ls plugins/ghidra_12.1.2_PUBLIC/support/analyzeHeadless
```

NOTE: Ghidra requires Java 17+.

3.4 impacket (Registry Hash Extraction)

```
pip install impacket  
impacket-secretsdump --help # verify
```

4. System Tool Installation - Windows

Download and install each tool, then add its folder to your system PATH:

Tool	Download URL	Install Notes
exiftool	https://exiftool.org	Rename exiftool(-k).exe to exiftool.exe
tshark	https://www.wireshark.org	Included in Wireshark installer
john	https://www.openwall.com/john/	Add the run\ folder to PATH
steghide	https://steghide.sourceforge.net	Add install folder to PATH
SleuthKit	https://www.sleuthkit.org/sleuthkit/	Add bin\ folder to PATH
Volatility3	pip install volatility3	Adds vol.exe to .venv\Scripts\
Java 17	https://adoptium.net/	Required for Ghidra

5. MobSF Setup (Mobile Security Framework)

MobSF performs deep static and dynamic analysis of Android APKs. It runs as a separate server on port 8001.

5.1 Start MobSF Server

```
cd plugins/Mobile-Security-Framework-MobSF

# First-time setup (Linux/macOS):
./setup.sh

# Start server:
python manage.py runserver 0.0.0.0:8001

# Windows:
setup.bat
python manage.py runserver 0.0.0.0:8001
```

NOTE: MobSF must be running BEFORE submitting APK scans from the CyberX dashboard.

5.2 Configure the API Key

1. Open <http://localhost:8001> in your browser
2. Go to Settings -> REST API -> copy the key
3. Paste it into the wrapper:

```
# backend/wrappers/mobsf_wrapper.py
MOBSF_URL      = "http://127.0.0.1:8001"
MOBSF_API_KEY  = "paste-your-api-key-here"
```

5.3 Static Analysis Pipeline (7 Steps)

Step	Name	Description
1	Connectivity Check	Verifies MobSF is reachable before uploading (fail-fast to avoid long waits)
2	Upload APK	Uploads the APK via REST API; MobSF returns a unique file hash
3	Trigger Scan	Starts the MobSF static analysis engine using the file hash
4	Fetch JSON Report	Downloads the full analysis report (manifest, code, cert, permissions, APKID)
5	Feature Extraction	Extracts 11 risk dimensions: severity, permissions, APKID, malware, network, components, certificate, secrets, tra
6	Risk Score Calc	Each dimension gets a 0-100 score; a weighted final score is computed
7	PDF Report Download	MobSF PDF report downloaded and returned as base64 for browser display

5.4 Risk Scoring Weights

Feature Dimension	Weight	Max Contribution
Severity Findings (High/Medium/Low)	30%	High=25pts, Medium=10pts, Low=3pts each
Permission Combinations	20%	Malicious patterns: SMS malware, spyware, etc
Malware Detection (VirusTotal)	30%	Log-scaled from VT positives count
APKID / Anti-Analysis Techniques	15%	Packer=15-25pts, Anti-VM/Debug=20pts each
Network Security	15%	Cleartext=20, CertPinBypass=25, TrustUserCA
Certificate Issues	15%	Issues=15pts each, Expired=40pts
Exported Components	10%	5pts per exported activity/service/receiver
Hardcoded Secrets	10%	15pts per found secret

Manifest Flags	10%	Debuggable=20, Backup=15, No Secure Flag=
----------------	-----	---

5.5 Risk Classification Thresholds

Score Range	Risk Level	Action Required
75 - 100	CRITICAL RISK	Immediate investigation - likely malware
60 - 74	HIGH RISK	Serious threats detected, do not distribute
40 - 59	MEDIUM RISK	Moderate vulnerabilities, review before use
20 - 39	LOW RISK	Minor issues, considered reasonably safe
0 - 19	MINIMAL RISK	App appears clean and safe

5.6 Malicious Permission Patterns Detected

Pattern Name	Permissions Required	Risk Weight
SMS Malware	READ_SMS + SEND_SMS + RECEIVE_SMS	+25 pts
Overlay Installer	SYSTEM_ALERT_WINDOW + REQUEST_INSTALL_PACKAGES	+20 pts
Spyware	READ_CONTACTS + READ_CALL_LOG + LOCATION + AUDIO	+22 pts
Financial Malware	SYSTEM_ALERT_WINDOW + READ_SMS + RECEIVE_SMS	+18 pts
Device Admin	BIND_DEVICE_ADMIN + WRITE_SETTINGS	+15 pts
Complete Control	READ_PHONE_STATE + ALERT_WINDOW + INSTALL + SETTINGS	+30 pts
Data Exfiltration	READ_CONTACTS + READ_SMS + LOCATION + STORAGE	+20 pts

6. All 15 Analysis Tools - API Reference

Endpoint	Parameters	Engine	Description
/analyze/exiftool	file_path	ExifTool	Metadata from any file (image, doc, video)
/analyze/yara	file_path, rules_path	YARA	Scan file against custom malware rules
/analyze/autopsy	image_path, scan_type	SleuthKit	Disk images: mmls/fsstat/fls/timeline
/analyze/ghidra	file_path	Ghidra	Headless binary reverse engineering
/analyze/androguard	file_path	Androguard	APK static: permissions, activities, score
/analyze/mobsf	file_path	MobSF Static	Deep APK analysis, 9-dimension risk score
/analyze/mobsf_dynamic	file_path, wait_time	MobSF Dyn.	Runtime APK analysis (needs emulator)
/analyze/volatility	file_path, scan_type	Volatility3	Memory: info/pslist/malfind/netscan
/analyze/threat_intel	target, scan_type, api_key	Multi-source	IP/domain: ip/whois/virustotal/shodan/...
/analyze/network	file_path, scan_type	tshark	PCAP: dns / hierarchy / endpoints
/analyze/email	file_path	eml-parser	Parse .eml: headers, attachments
/analyze/evtx	file_path	python-evtx	Windows event log EVTX parser
/analyze/stego	file_path	steghide	Hidden data in images/audio
/analyze/hash	file_path	John Ripper	Crack hash files with John the Ripper
/analyze/brute	mode, target, ...	Custom Eng.	Brute: http/ssh/ftp/zip/pdf/hash
/analyze/registry	sam_path, system_path	impacket+John	Extract & crack NTLM from SAM/SYSTEM

7. Running the Platform

7.1 Linux / macOS - Quick Start

```
# Option A: convenience script
chmod +x run_linux.sh && ./run_linux.sh

# Option B: manual
# Terminal 1 - Backend
cd backend
uv run --with uvicorn uvicorn main:app --reload --port 8000

# Terminal 2 - Frontend
cd frontend && npm run dev
```

7.2 Windows - Quick Start

```
# Double-click run_windows.bat
# OR manually in two terminals:

# Terminal 1 - Backend
cd backend
..\venv\Scripts\activate
uvicorn main:app --reload --port 8000

# Terminal 2 - Frontend
cd frontend && npm run dev
```

7.3 Service URLs

Service	URL
Frontend UI	http://localhost:5173
Backend API	http://localhost:8000
API Swagger Docs	http://localhost:8000/docs
Tool Diagnostics	http://localhost:8000/diagnostics
MobSF Server	http://localhost:8001

7.4 Verify All Tools via Diagnostics Endpoint

```
curl http://localhost:8000/diagnostics

# Returns:
# {
#   "os": "Linux",
#   "john": "/usr/sbin/john",
#   "exiftool": "/usr/bin/exiftool",
#   "tshark": "/usr/bin/tshark",
#   "steghide": "/usr/bin/steghide",
#   "volatility": ".venv/bin/vol",
#   "ghidra": "plugins/ghidra_12.1.2_PUBLIC/support/analyzeHeadless",
#   "secretsdump": ".venv/bin/impacket-secretsdump"
# }
```

8. One-Command Setup Scripts

Linux / macOS

```
chmod +x setup_linux.sh && ./setup_linux.sh
```

This script:

- Creates the Python virtual environment
- Installs all pip packages from requirements.txt
- Runs npm install for the frontend
- Checks each external tool and prints download links for any that are missing

Windows (Run as Administrator)

```
setup_windows.bat
```

Same as above for Windows. Opens two browser windows with the app after setup.